Sorting and Searching

A. Linear Search

1 second, 256 megabytes

You are given an array of integers and a number X.

Determine whether X exists in the array.

Input

The first line contains an integer n ($1 \le n \le 10^5$) — the size of the array.

The second line contains n integers A_i ($-10^9 \le A_i \le 10^9$).

The third line contains an integer X ($-10^9 \le X \le 10^9$).

Output

Print `YES` if X exists in the array, otherwise print `NO`.

input
5 1 3 7 9 2 7
output
YES

input
4 5 8 10 12 6
output
NO

B. Binary Search

1 second, 256 megabytes

You are given a sorted array of integers and a number X.

Determine whether X exists in the array.

Important: The array is sorted in non-decreasing order. Your solution is expected to use **binary search**.

Input

The first line contains an integer n ($1 \le n \le 10^5$) — the size of the array.

The second line contains n integers A_i ($-10^9 \le A_i \le 10^9$) in non-decreasing order.

The third line contains an integer X ($-10^9 \le X \le 10^9$).

Output

Print `YES` if X exists in the array, otherwise print `NO`.

input
5 1 3 5 7 9 7
output
YES

input
6 -5 -2 0 4 10 12 3
output
NO

C. Binary Search (Strings)

1 second, 256 megabytes

You are given a guest list containing names in sorted lexicographical order.

Given the name of a guest, determine whether the name exists in the list.

Important: The list is sorted. Your solution is expected to use **binary search**.

Input

The first line contains an integer n $(1 \le n \le 10^5)$ — the number of names.

The next n lines each contain a name S_i.

Each name:

- consists only of lowercase English letters,
- has length between 1 and 20.

The names are given in non-decreasing lexicographical order.

The last line contains a name X — the guest to search for.

Output

Print `YES` if the name exists in the list, otherwise print `NO`.

input
5 alice bob charlie david emma charlie

output
YES

input
3 anna bella carol diana

output
NO

D. Selection Sort Trace

1 second, 256 megabytes

You are given an array of N integers. Your task is to simulate the **Selection Sort** algorithm and print the trace of each pass.

In Selection Sort, during the i-th pass $(1 \le i \le N-1)$:

- Find the minimum element from the unsorted part of the array.
- Swap it with the element at index i.
- Print the array after the swap, along with the element selected as minimum.

Format:

```
Pass i: array_after_swap , min_selected = x
```

Input

The first line contains a single integer N $(1 \le N \le 500)$.

The second line contains N integers $A_1, A_2, \ldots, A_N$.

Output

For each pass i $(1 \le i \le N-1)$, print as required.

input
6 30 10 50 20 40 60
output
Pass 1: 10 30 50 20 40 60 , min_selected = 10 Pass 2: 10 20 50 30 40 60 , min_selected = 20 Pass 3: 10 20 30 50 40 60 , min_selected = 30 Pass 4: 10 20 30 40 50 60 , min_selected = 40 Pass 5: 10 20 30 40 50 60 , min_selected = 50

Example 1, Pass 1 explanation:

The minimum element in [30,10,50,20,40,60] is 10.

Swap it with the first element.

New array: [10,30,50,20,40,60]

E. Bubble Sort Trace

1 second, 256 megabytes

You are given an array of N integers. Your task is to simulate the **Bubble Sort** algorithm and print the trace of each pass.

In Bubble Sort:

- Compare adjacent elements and swap them if they are in the wrong order.
- After each pass, print the array and the number of swaps in that pass.
- Stop early if a pass has 0 swaps.

Format

```
Pass i: array_after_swap , swaps = x
```

Input

The first line contains a single integer N $(1 \le N \le 500)$ — the size of the array. The second line contains N integers $A_1, A_2, \ldots, A_N$.

Output

For each pass i $(1 \le i \le N-1)$, print the array after the pass and the number of swaps in that pass. Stop if swaps = 0.

input
5 5 1 4 2 8
output
Pass 1: 1 4 2 5 8 , swaps = 3 Pass 2: 1 2 4 5 8 , swaps = 1 Pass 3: 1 2 4 5 8 , swaps = 0

Example 1, Pass 1 explanation:

Compare and swap adjacent elements in [5,1,4,2,8]. After the first pass, array = [1,4,2,5,8], swaps = 3.

F. Insertion Sort Trace

1 second, 256 megabytes

You are given an array of N integers. Your task is to simulate the **Insertion Sort** algorithm and print the trace after inserting each element.

In Insertion Sort:

- Insert each element into its correct position in the sorted portion.
- After inserting an element, print the array, highlighting the sorted portion using '|' to separate sorted and unsorted parts.
- Print the number of shifts required to insert that element.

Format:

```
Pass i: array_after_insertion, sorted portion | unsorted portion, shifts = x
```

Input
The first line contains a single integer N $(1 \le N \le 500)$ — the size of the array.

The second line contains N integers $A_1, A_2, \ldots, A_N$.

Output
For each insertion pass i $(1 \le i \le N-1)$, print the array after inserting the i-th element, the sorted portion separated by '|', and the number of shifts required.

input
5 5 1 4 2 3
output
Pass 1: 1 5 4 2 3 , 1 5 | 4 2 3 , shifts = 1 Pass 2: 1 4 5 2 3 , 1 4 5 | 2 3 , shifts = 1 Pass 3: 1 2 4 5 3 , 1 2 4 5 | 3 , shifts = 2 Pass 4: 1 2 3 4 5 , 1 2 3 4 5 | , shifts = 2

Example 1, Pass 1 explanation:

We take the second element, 1, and insert it into the sorted portion consisting of the first element [5].

Compare 1 with 5: since 1 < 5, shift 5 one position to the right.

Place 1 in the first position.

The array after this insertion is [1,5,4,2,8]. Number of shifts required = 1.

G. Selection Sort v/s Insertion Sort

1 second, 256 megabytes

You are given an array of n integers.

We want to compare two sorting algorithms based on how much "work" they do:

- **Insertion Sort:** Work is measured as the number of **shifts**. A shift happens when an element is moved one position to the right to make space for inserting another element. Comparisons are not counted.
- **Selection Sort (optimized):** Work is measured as the number of **swaps**. A swap happens only if the minimum element found is different from the current element. If the minimum is already at the current position, no swap is counted.

Determine which algorithm performs better (fewer operations) on the given array.

- Print `Insertion Sort` if insertion sort does fewer shifts.
- Print `Selection Sort` if selection sort does fewer swaps.
- Print `Tie` if both perform equally.

Input
The first line contains t $(1 \le t \le 100)$ – the number of test cases. For each test case:

- The first line contains n $(1 \le n \le 5000)$.
- The second line contains n integers $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le 10^9)$.

It is guaranteed that the sum of n over all test cases does not exceed 5000.

Output
For each testcase, output which algorithm is better, or whether it is a tie.

input
4 5 1 2 3 4 5 5 5 4 3 2 1 4 3 1 4 2 3 2 1 3

output

```
Tie
Selection Sort
Tie
Tie
```

Test Case 1: [1, 2, 3, 4, 5]

Selection Sort Trace:

- Pass 1: swaps = 0, array = [1, 2, 3, 4, 5]
- Pass 2: swaps = 0, array = [1, 2, 3, 4, 5]
- Pass 3: swaps = 0, array = [1, 2, 3, 4, 5]
- Pass 4: swaps = 0, array = [1, 2, 3, 4, 5]

Total swaps = 0
Insertion Sort Trace:

- Pass 1: shifts = 0, array = [1, 2, 3, 4, 5]
- Pass 2: shifts = 0, array = [1, 2, 3, 4, 5]
- Pass 3: shifts = 0, array = [1, 2, 3, 4, 5]
- Pass 4: shifts = 0, array = [1, 2, 3, 4, 5]

Total shifts = 0
Result: **Tie**

—

Test Case 2: [5, 4, 3, 2, 1]

Selection Sort Trace:

- Pass 1: swaps = 1, array = [1, 4, 3, 2, 5]
- Pass 2: swaps = 1, array = [1, 2, 3, 4, 5]
- Pass 3: swaps = 0, array = [1, 2, 3, 4, 5]
- Pass 4: swaps = 0, array = [1, 2, 3, 4, 5]

Total swaps = 2
Insertion Sort Trace:

- Pass 1: shifts = 1, array = [4, 5, 3, 2, 1]
- Pass 2: shifts = 2, array = [3, 4, 5, 2, 1]
- Pass 3: shifts = 3, array = [2, 3, 4, 5, 1]
- Pass 4: shifts = 4, array = [1, 2, 3, 4, 5]

Total shifts = 10
Result: **Selection Sort**

—

Test Case 3: [3, 1, 4, 2]

Selection Sort Trace:

- Pass 1: swaps = 1, array = [1, 3, 4, 2]
- Pass 2: swaps = 1, array = [1, 2, 4, 3]
- Pass 3: swaps = 1, array = [1, 2, 3, 4]

Total swaps = 3
Insertion Sort Trace:

- Pass 1: shifts = 1, array = [1, 3, 4, 2]
- Pass 2: shifts = 0, array = [1, 3, 4, 2]
- Pass 3: shifts = 2, array = [1, 2, 3, 4]

Total shifts = 3
Result: **Tie**

—

Test Case 4: [2, 1, 3]

Selection Sort Trace:

- Pass 1: swaps = 1, array = [1, 2, 3]
- Pass 2: swaps = 0, array = [1, 2, 3]

Total swaps = 1
Insertion Sort Trace:

- Pass 1: shifts = 1, array = [1, 2, 3]
- Pass 2: shifts = 0, array = [1, 2, 3]

Total shifts = 1
Result: **Tie**

H. Merge Two Sorted Arrays

1 second, 256 megabytes

You are given two arrays of integers, both sorted in non-decreasing order.

Merge the two arrays into a single sorted array.

Input

The first line contains two integers n and m ($1 \le n, m \le 10^5$) — the sizes of the arrays.

The second line contains n integers A_i ($-10^9 \le A_i \le 10^9$).

The third line contains m integers B_i ($-10^9 \le B_i \le 10^9$).

Both arrays are sorted in non-decreasing order.

Output

Print the merged sorted array.

input
3 4 1 3 5 2 4 6 8
output
1 2 3 4 5 6 8

input
2 3 -5 10 -6 0 12
output
-6 -5 0 10 12